

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnatura Universitaria en Programación (TUP)

Programación I

Gestión de Datos de Países

Trabajo Práctico Integrador

Estudiantes:

- **Nehuen Aumedes Diez**
- **Lucía Caramuti**

Docentes:

- **Florencia Gubiotti**
- **Franco Gonzalez**

Fecha de entrega: Junio 2026

ÍNDICE

Introducción	2
Marco Teórico	2
Condicionales en Python	2
Estructuras Repetitivas en Python	3
Estructuras de datos: listas y diccionarios	5
Ordenamiento de Datos en Python	7
Estadísticas Básicas en Python	8
Archivos CSV y persistencia	8
Funciones	9
Manejo de errores y validación de entrada	9
Decisiones Técnicas y Arquitectura	10
Diagrama de flujo del menú principal	10
Estructura modular del código	10
Ejemplos de ejecución	11
Dificultades y Conclusiones	14
Bibliografía	15
Links del Proyecto	15

1. INTRODUCCIÓN

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar una aplicación en Python para la gestión de datos de países. El sistema permite realizar operaciones de búsqueda, filtrado, ordenamiento y generación de estadísticas a partir de información almacenada en un archivo CSV.

Durante el desarrollo se aplicaron conceptos fundamentales de Programación I, tales como listas, diccionarios, funciones, estructuras condicionales y repetitivas, lectura de archivos y modularización del código.

1. MARCO TEORICO

Condicionales en Python

Los condicionales son estructuras de control que permiten que un programa tome decisiones según se cumplan o no determinadas condiciones. Gracias a ellos, es posible ejecutar diferentes acciones dependiendo del resultado de una evaluación lógica.

En Python, las estructuras condicionales principales son if, elif y else.

La sentencia if se utiliza para evaluar una condición. Si dicha condición es verdadera, se ejecuta el bloque de código asociado. Cuando se desea evaluar una segunda condición en caso de que la primera no se cumpla, se utiliza elif (abreviatura de "else if"). Finalmente, la sentencia else permite ejecutar un bloque de código cuando ninguna de las condiciones anteriores resulta verdadera.

Los condicionales suelen trabajar junto con operadores de comparación como:

- Igual que (==)
- Distinto de (!=)
- Mayor que (>)
- Menor que (<)
- Mayor o igual que (>=)
- Menor o igual que (<=)

También pueden combinarse utilizando operadores lógicos como and, or y not.

En este trabajo práctico, los condicionales fueron fundamentales para controlar el funcionamiento del sistema. Se utilizaron para validar los datos ingresados por el usuario, verificar la existencia de países dentro de la base de datos, controlar las opciones seleccionadas en el menú y evitar errores durante la ejecución del programa.

Por ejemplo, al agregar un país se puede comprobar que el usuario no deje campos vacíos:

```
if nombre == "":
    print("Error: debe ingresar un nombre de país.")
else:
    agregar_pais()
```

Asimismo, los condicionales permiten verificar si una búsqueda obtuvo resultados o si un país pertenece a un determinado continente para aplicar filtros específicos.

El uso de estas estructuras permitió que el sistema tomara decisiones de manera automática, mejorando la validación de datos y la interacción con el usuario.

Estructuras Repetitivas en Python

Las estructuras repetitivas, también conocidas como bucles o ciclos, permiten ejecutar un bloque de código varias veces mientras se cumpla una determinada condición o para recorrer una colección de elementos. Su principal ventaja es evitar la repetición innecesaria de instrucciones, haciendo que los programas sean más eficientes y fáciles de mantener.

En Python, las estructuras repetitivas más utilizadas son for y while.

Bucle for

El bucle for se utiliza para recorrer secuencias de datos, como listas, cadenas de texto o diccionarios. En cada iteración, la variable de control toma el valor de un elemento de la secuencia hasta recorrerla completamente.

Por ejemplo:

```
países = ["Argentina", "Brasil", "Chile"]

for país in países:
    print(país)
```

Este código muestra cada país almacenado en la lista.

Bucle while

El bucle while ejecuta un bloque de instrucciones mientras una condición sea verdadera. Antes de cada repetición, el programa evalúa la condición y continúa ejecutándose hasta que esta deje de cumplirse.

Por ejemplo:

```
contador = 1
```

```
while contador <= 5:
    print(contador)
    contador += 1
```

En este caso, el programa muestra los números del 1 al 5.

Aplicación en el Trabajo Práctico

En este proyecto se utilizaron ambas estructuras repetitivas para gestionar la información de los países.

El bucle for se empleó para recorrer la lista de países y realizar operaciones como búsquedas, filtros, ordenamientos y generación de estadísticas.

Por ejemplo:

```
for pais in paises:
    if pais["continente"] == "América":
        print(pais["nombre"])
```

Por otro lado, el bucle while se utilizó para mantener activo el menú principal del sistema hasta que el usuario decidiera finalizar la ejecución.

Por ejemplo:

```
while opcion != 7:
    mostrar_menu()
```

Gracias al uso de estas estructuras repetitivas, el programa puede procesar grandes cantidades de información de manera automática y eficiente, facilitando la interacción con el usuario y la gestión de los datos almacenados.

Estructuras de datos: listas y diccionarios

Las listas son una de las estructuras de datos más utilizadas en Python. Permiten almacenar varios elementos dentro de una misma variable y pueden contener distintos tipos de datos, como números, cadenas de texto o incluso otras listas.

Una característica importante de las listas es que son mutables, es decir, sus elementos pueden modificarse, agregarse o eliminarse después de haber sido creadas.

Las listas se representan mediante corchetes ([]), con los elementos separados por comas. Por ejemplo:

```
países = ["Argentina", "Brasil", "Chile"]
```

Python ofrece diversos métodos para trabajar con listas. Entre los más utilizados se encuentran:

- `append()`: agrega un elemento al final de la lista.
- `insert()`: inserta un elemento en una posición específica.
- `remove()`: elimina un elemento determinado.
- `pop()`: elimina y devuelve un elemento de la lista.
- `sort()`: ordena los elementos.
- `reverse()`: invierte el orden de los elementos.
- `count()`: cuenta cuántas veces aparece un valor.
- `index()`: devuelve la posición de un elemento dentro de la lista.

Las listas resultan especialmente útiles para almacenar conjuntos de datos relacionados y permiten recorrer sus elementos mediante estructuras repetitivas como los bucles `for` y `while`.

En este trabajo práctico se utilizaron listas para almacenar la información de los países cargados desde el archivo CSV y para realizar operaciones de búsqueda, filtrado, ordenamiento y generación de estadísticas.

Diccionarios en Python

Los diccionarios son una estructura de datos que permite almacenar información mediante pares de clave y valor. A diferencia de las listas, donde los elementos se acceden a través de una posición numérica, en los diccionarios se accede a los datos utilizando una clave única.

Los diccionarios se representan con llaves `{}` y cada elemento está formado por una clave y un valor separados por dos puntos. Por ejemplo:

```
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América"
}
```

En este caso, las claves son "nombre", "poblacion", "superficie" y "continente", mientras que los valores corresponden a la información almacenada para cada una de ellas.

Entre las principales operaciones que pueden realizarse con un diccionario se encuentran agregar nuevos datos, modificar valores existentes, eliminar elementos y consultar información a través de sus claves.

Algunos métodos y operaciones comunes son:

- Acceder a un valor mediante una clave.
- Utilizar `get()` para obtener un valor de forma segura.
- Agregar nuevos pares clave-valor.
- Eliminar elementos con `del`.
- Recorrer el diccionario mediante estructuras repetitivas.

- Obtener las claves o los valores almacenados.

Los diccionarios son especialmente útiles cuando se necesita organizar información relacionada entre sí, ya que permiten identificar cada dato mediante un nombre descriptivo en lugar de una posición numérica.

En este trabajo práctico se utilizaron diccionarios para representar la información de cada país. Cada país se almacena como un diccionario que contiene su nombre, población, superficie y continente. Posteriormente, estos diccionarios se guardan dentro de una lista para poder realizar búsquedas, filtros, ordenamientos y cálculos estadísticos sobre los datos cargados desde el archivo CSV.

Por ejemplo:

```
{
  "nombre": "Brasil",
  "poblacion": 213993437,
  "superficie": 8515767,
  "continente": "América"
}
```

Gracias al uso de diccionarios, la información puede organizarse de manera clara y accederse fácilmente a cada dato mediante su clave correspondiente

Ordenamiento de Datos en Python

El ordenamiento de datos es un proceso que permite organizar la información siguiendo un criterio determinado, ya sea de forma ascendente o descendente. Esta técnica facilita la búsqueda, comparación y análisis de los datos almacenados en una estructura.

Python ofrece diferentes herramientas para ordenar elementos. Entre las más utilizadas se encuentran la función `sorted()` y el método `sort()`.

La función `sorted()` genera una nueva lista ordenada sin modificar la lista original, mientras que el método `sort()` ordena directamente los elementos de la lista existente.

Además, Python permite personalizar el criterio de ordenamiento mediante el parámetro `key`, el cual indica qué atributo o valor debe utilizarse para realizar la comparación entre los elementos.

En este trabajo práctico, el ordenamiento se utilizó para organizar los países según distintos criterios, como nombre, población o superficie. También fue posible mostrar los resultados tanto en orden ascendente como descendente, facilitando el análisis de la información almacenada.

Por ejemplo, para ordenar los países por población se puede utilizar:

```
países.sort(key=lambda pais: pais["poblacion"])
```

El uso de técnicas de ordenamiento permitió presentar la información de manera más clara y mejorar la experiencia del usuario al consultar los datos del sistema.

Estadísticas Básicas en Python

Las estadísticas básicas permiten obtener información relevante a partir de un conjunto de datos. Estas herramientas facilitan el análisis y la interpretación de la información almacenada en un sistema.

Python dispone del módulo `statistics`, que incluye funciones para calcular diferentes medidas estadísticas, como la media, la mediana y la moda.

La media (`mean`) representa el promedio de un conjunto de valores. La mediana (`median`) corresponde al valor central cuando los datos se encuentran ordenados. Por su parte, la moda (`mode`) indica el valor que aparece con mayor frecuencia dentro de un conjunto de datos.

En este trabajo práctico se implementaron estadísticas relacionadas con los países almacenados en el sistema. Entre ellas se encuentran el cálculo del promedio de población, el promedio de superficie, la identificación del país con mayor y menor población, y la cantidad de países registrados por continente.

Estas estadísticas permiten obtener una visión general de los datos almacenados y facilitan la toma de decisiones a partir de la información procesada.

Archivos CSV y persistencia de datos

El formato CSV (Comma Separated Values) se utilizó para almacenar la información de los países de manera sencilla y organizada. Este tipo de archivo permite guardar datos en forma de tabla, donde cada fila representa un registro y cada columna un dato específico.

Para trabajar con estos archivos se utilizó el módulo `csv` de Python, que forma parte de la biblioteca estándar del lenguaje y permite leer y escribir archivos CSV de manera simple.

En el proyecto se utilizaron las siguientes herramientas:

Clase	Uso en el proyecto
<code>csv.DictReader</code>	se uso para leer el archivo CSV y convertir cada fila en un diccionario, facilitando el acceso a los datos mediante claves como nombre, población, superficie y continente.
<code>csv.DictWriter</code>	se uso para guardar la información de los países en el archivo CSV manteniendo los encabezados y la estructura de los datos.

Además, se empleó la codificación `utf-8-sig` para asegurar que los caracteres especiales del idioma español, como tildes y la letra ñ, se visualicen correctamente en distintos sistemas operativos.

Funciones

Las funciones son bloques de código que permiten organizar un programa en tareas específicas. Su principal objetivo es evitar repetir código y hacer que los programas sean más ordenados, fáciles de entender y mantener.

Una función se define utilizando la palabra reservada `def`, seguida del nombre de la función y los parámetros que necesita para realizar su tarea. Cuando una función es llamada, ejecuta las instrucciones que contiene y puede devolver un resultado mediante la instrucción `return`.

Los parámetros permiten que una misma función pueda trabajar con diferentes datos. Además, las variables creadas dentro de una función son variables locales, por lo que solo pueden utilizarse dentro de ella.

En Python también es posible documentar las funciones mediante comentarios especiales llamados *docstrings*, los cuales sirven para explicar qué hace cada función y facilitar la comprensión del código.

En este trabajo práctico se utilizaron funciones para dividir el programa en módulos más pequeños y organizados. Cada función tiene una responsabilidad específica, lo que permite que el sistema sea más claro y fácil de mantener.

Por ejemplo, se implementaron funciones para:

- Cargar los datos desde el archivo CSV.
- Agregar un nuevo país.
- Actualizar la población o superficie de un país.
- Buscar países por nombre.
- Filtrar países por continente, población o superficie.
- Ordenar los datos según distintos criterios.
- Calcular estadísticas como el promedio de población o la cantidad de países por continente.

Un ejemplo simplificado de una función utilizada en el proyecto podría ser:

```
def buscar_pais(paises, nombre):
    for pais in paises:
        if pais["nombre"].lower() == nombre.lower():
            return pais
    return None
```

Esta función recibe la lista de países y el nombre que se desea buscar. Si encuentra el país, devuelve toda su información; en caso contrario, devuelve `None`.

El uso de funciones permitió que el código fuera más modular, reutilizable y fácil de comprender, siguiendo las buenas prácticas de programación solicitadas en la consigna.

Manejo de errores y validación de entrada

El manejo de excepciones con bloques **try/except** permite que el programa responda de forma controlada ante errores en tiempo de ejecución, como datos inválidos en el CSV o fallas al abrir archivos. Python diferencia entre *ValueError* (tipo de dato incorrecto) y *OSError* (error del sistema operativo al acceder a archivos).

El módulo **os** se emplea para verificar la existencia del archivo CSV antes de intentar leerlo (*os.path.exists*) y para obtener la ruta absoluta del script (*os.path.dirname* / *os.path.abspath*), asegurando que el archivo se ubique siempre junto al código fuente independientemente del directorio de trabajo.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

Diagrama de flujo del menú principal

A continuación se presenta el diagrama de flujo que describe el ciclo de vida de la aplicación, desde la carga inicial del CSV hasta la selección y ejecución de cada operación disponible en el menú:



Estructura modular del código

PERSISTENCIA

`cargar_csv()` `guardar_csv(paises)`

Lectura y escritura del archivo CSV. Encapsulan completamente el acceso al disco.

VALIDACIÓN

pedir_entero(msg, min, max) pedir_texto(msg) pedir_opcion(msg, opciones)

Garantizan que el usuario ingrese datos válidos antes de procesarlos.

PRESENTACIÓN

imprimir_separador() imprimir_pais() imprimir_lista_paises()

Formatean y muestran información de manera consistente y legible.

OPERACIONES

*agregar_pais() actualizar_pais() buscar_por_nombre() filtrar_paises() ordenar_paises()
mostrar_estadisticas()*

Lógica de negocio principal. Cada función opera sobre la lista en memoria.

Ejemplos de ejecución**Ejemplo 1 — Agregar un país:**

```

GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
2. Actualizar población y superficie de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Listar todos los países
0. Salir

Elegí una opción: 4

— Filtrar países —
1. Por continente
2. Por rango de población
3. Por rango de superficie
Elegí una opción: 1
Nombre del continente:
[!] El campo no puede estar vacío.
Nombre del continente: asdasd

Países en Asdasd (0 país/es encontrado/s)

No se encontraron países.

```

Ejemplo 2 — Estadísticas del dataset:

```
GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
2. Actualizar población y superficie de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Listar todos los países
0. Salir

Elegí una opción: 6

— Estadísticas —

POBLACIÓN
Mayor : China (1.410.000.000)
Menor : Uruguay (3.500.000)
Promedio: 150.777.500
.....
SUPERFICIE
Mayor : Rusia (17.098.200 km²)
Menor : Corea del Sur (100.000 km²)
Promedio: 2.330.030 km²
.....
PAÍSES POR CONTINENTE
América      : 12
Asia         : 12
Europa       : 10
África       :  4
Oceania      :  2

Total de países en el dataset: 40

Presioná Enter para continuar...
```

Ejemplo 3 — Filtrar países:

GESTIÓN DE DATOS DE PAÍSES

1. Agregar país
 2. Actualizar población y superficie de un país
 3. Buscar país por nombre
 4. Filtrar países
 5. Ordenar países
 6. Mostrar estadísticas
 7. Listar todos los países
 0. Salir
-

Elegí una opción: 4

— Filtrar países —

1. Por continente
2. Por rango de población
3. Por rango de superficie

Elegí una opción: 3

Ingresá el rango de superficie en km²:

Mínimo (km²): a

[!] Ingresá un número entero válido (sin decimales ni letras).

Mínimo (km²): 23

Máximo (km²): !

[!] Ingresá un número entero válido (sin decimales ni letras).

Máximo (km²): 966464

Superficie entre 23 y 966.464 km² (22 país/es encontrado/s)

1. Chile

Continente : América

Población : 19.000.000

Superficie : 756.000 km²

2. España

Continente : Europa

Población : 47.000.000

Superficie : 505.000 km²

3. Francia

Continente : Europa

Población : 67.000.000

Superficie : 643.000 km²

3. DIFICULTADES Y CONCLUSIONES

Obstáculos técnicos encontrados

Codificación de caracteres (utf-8-sig)	Al ejecutar el programa en Windows, los caracteres especiales como tildes y la ñ generaban errores de lectura. Se resolvió usando la codificación utf-8-sig tanto en la lectura como en la escritura del CSV, que maneja correctamente el BOM (Byte Order Mark) de Windows.
Validación robusta de entradas	Garantizar que el usuario solo ingrese datos válidos (enteros positivos, texto sin números, opciones dentro del rango) requirió diseñar funciones de validación con bucles while y bloques try/except anidados. La función pedir_texto() tuvo que manejar por separado el caso de cadenas vacías y el de texto con números o símbolos.
Ruta del archivo CSV	Ejecutar el programa desde distintos directorios hacía que el CSV no se encontrara. Se solucionó usando <code>os.path.dirname(os.path.abspath(__file__))</code> para siempre construir la ruta relativa a la ubicación del script, no al directorio de trabajo.

Conclusiones

La realización de este trabajo práctico integrador permitió consolidar los conocimientos adquiridos durante la cursada de Programación I.

A través del desarrollo del sistema se aplicaron conceptos teóricos en una situación práctica, fortaleciendo la comprensión del lenguaje Python y de las principales estructuras de programación.

Además, el proyecto permitió desarrollar habilidades relacionadas con la organización del código, la resolución de problemas y el trabajo colaborativo. La experiencia resultó enriquecedora, ya que permitió comprender cómo los distintos conceptos estudiados pueden combinarse para construir una aplicación funcional y útil.

Una de las mayores dificultades que encontramos fue integrar varios conceptos nuevos al mismo tiempo. Aprender a utilizar GitHub, trabajar con archivos CSV y aplicar la modularización del código representó un desafío importante, ya que debimos comprender cómo se relacionaban estas herramientas entre sí dentro de un mismo proyecto. Sin embargo, a medida que avanzamos en el desarrollo y realizamos pruebas, logramos comprender mejor su funcionamiento y aplicarlos correctamente en nuestro sistema.

4. BIBLIOGRAFÍA / WEBGRAFÍA

- 🚩 Python Software Foundation. (s.f.). *Built-in types — Lists*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/library/stdtypes.html#typeseq-list>
- 🚩 Python Software Foundation. (s.f.). *Dictionaries*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/tutorial/datastructures.html#dictionaries>
- 🚩 Python Software Foundation. (s.f.). *Function definitions*. Python Documentation. Recuperado el 13 de junio de 2026, de https://docs.python.org/3/reference/compound_stmts.html#function-definitions
- 🚩 Python Software Foundation. (s.f.). *Defining functions*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- 🚩 Python Software Foundation. (s.f.). *If statements*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/tutorial/controlflow.html#if-statements>
- 🚩 Python Software Foundation. (s.f.). *For statements*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/tutorial/controlflow.html#for-statements>
- 🚩 Python Software Foundation. (s.f.). *The while statement*. Python Documentation. Recuperado el 13 de junio de 2026, de https://docs.python.org/3/reference/compound_stmts.html#the-while-statement
- 🚩 Python Software Foundation. (s.f.). *Sorting HOW TO*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/howto/sorting.html>
- 🚩 Python Software Foundation. (s.f.). *statistics — Mathematical statistics functions*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/library/statistics.html>
- 🚩 Python Software Foundation. (s.f.). *csv — CSV file reading and writing*. Python Documentation. Recuperado el 13 de junio de 2026, de <https://docs.python.org/3/library/csv.html>

5. LINKS DEL PROYECTO

Repositorio GitHub:

<https://github.com/caramuti-lu/TPI-Programacion1-Gestion-Paises.git>

Video explicativo:

<https://www.youtube.com/watch?v=n0eDSiPMCzY>

